From Monovalent Dev to Bivalent DevOps to Trivalent DevIntOps

Intelligence as the third continuous loop in software engineering

Pavel Chertorogov · DRAFT · 18 October 2026

Canonical version: https://devintops.org/v1/ · License: CC BY 4.0

This is a draft, open for comments. It will change before version 1.0, which keeps this address. See the version history.

Abstract

Many agentic SDLC proposals put an AI agent into every box of the DevOps figure-eight. That can make each box faster. It doesn't, by itself, carry experience from one task to the next: a failed check is retried until it passes, and the next task starts from the same defaults. DevOps has long valued feedback and continual learning. This paper makes that learning an explicit, persistent and measured third loop next to Development and Operations. Intelligence, with the stages Understand, Predict, Orchestrate and Learn and with memory and judgment at its core, turns outcomes into lessons that change how each stage decides. Those outcomes include escaped bugs and incidents, but also review feedback, wasted work, cost and changes that went well, and deciding to change nothing is a valid result. The same logical pattern applies to the engineering system that builds and runs software and to the decisions a product makes for its users. It is a step toward self-improving software systems, with a bound: the loop aims to make later decisions better, while people keep the goals, the permissions and the scoring. A chemistry metaphor frames the shift: monovalent Dev, bivalent DevOps, trivalent DevIntOps. The paper defines the loop, its unit of work, its evidence and metrics, its owner and trust limits, lists the ways it can fail, says what result would show it wrong, and places it among related ideas.

The pattern in agentic SDLC proposals

Most agentic SDLC diagrams I have seen share one shape: the DevOps figure-eight with an AI agent in every box. A coding agent writes, a review agent reviews, a validation agent tests, a release agent ships and an operations agent watches. When a check fails, the work goes back until it passes.

That is useful. It is also the same loop, only faster.

This is not a claim that DevOps never learns. Its Third Way asks for continual experimentation and learning [1], and good teams already keep lessons as regression tests, alerts, runbooks and postmortem actions. The gap is narrower: in these diagrams, nothing makes that learning an explicit, persistent and measured loop. Three details show it.

- **The loops retry; they don't learn.** Retrying converges one task. Unless the outcome changes something the next task reads, task N+1 starts with the same context as task N, or with whatever someone curated by hand.

- **The hard parts are written down as assumptions:** that people keep the agents' context up to date, and that validation catches nearly everything. Those are exactly the things a third loop should produce: context learned from outcomes, and checks that grow, shrink and retire with the evidence.
- **Learning is drawn as an arrow,** usually from production back to the next spec. An arrow has no owner, no memory and no metric.

DORA's 2025 research describes AI as an amplifier of an organization's existing strengths and weaknesses [2]. That is a reason to look at the loop around the boxes, not only at the boxes.

Valence: Dev, DevOps, DevIntOps

In chemistry, valence is the number of bonds an atom can form. Two bonds only get you chains and rings. Branching starts at three.

Here valence is a metaphor, not a law: it counts the continuous loops bonded to one system. By that count, software engineering has added a valence twice already:

- **Monovalent, Dev.** Plan, code, build, test, hand it over. The question is "does it work?"
- **Bivalent, DevOps.** Development and operations bonded into one figure-eight: two loops, one system. You build it, you run it. The question becomes "does it keep working in production?"
- **Trivalent, DevIntOps.** A third loop, Intelligence, bonded to the same system. The question becomes "does it decide better, task after task?"

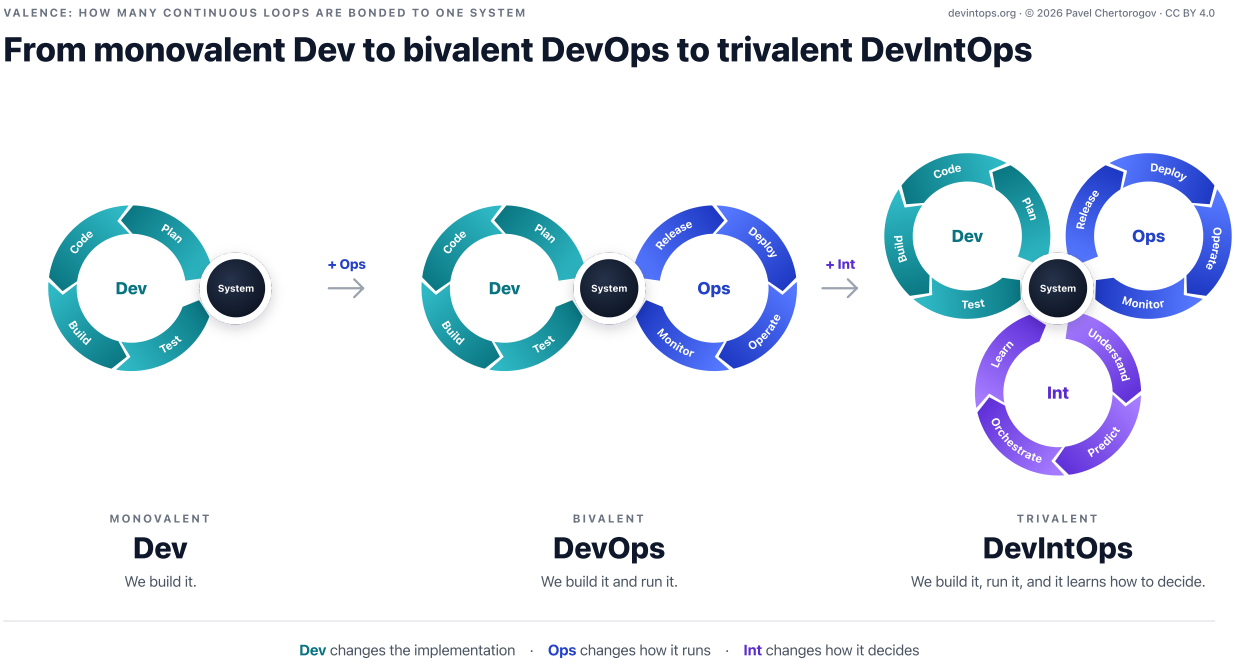


Figure 1. Valence at the system level, used as a metaphor: one, two and three continuous loops bonded to one system.

Each step keeps everything before it. DevOps didn't remove development, and DevIntOps doesn't remove DevOps. Putting an AI agent into every DevOps box speeds up the second step. It takes the third only if experience persists, reaches later decisions and gets checked against outcomes.

Figure 2 counts something else: the bonds of a single stage, not the loops around the system. A bond is a way one stage's outcome can change another stage's next run. In a simplified pipeline each stage bonds only to its neighbors, so you get chains (pipelines) and rings (retry until it passes). The retry arrow is a bond too, but it dies with the task. A third bond, to shared memory, outlives it.

Topology decides what can travel. In a chain, a lesson from production crawls back through every step, usually by way of someone writing the next spec. In a network it goes straight to the stage that can use it. Real pipelines branch more than this drawing does, and many already share some lessons across stages; the point is where lessons can travel, not the exact graph.

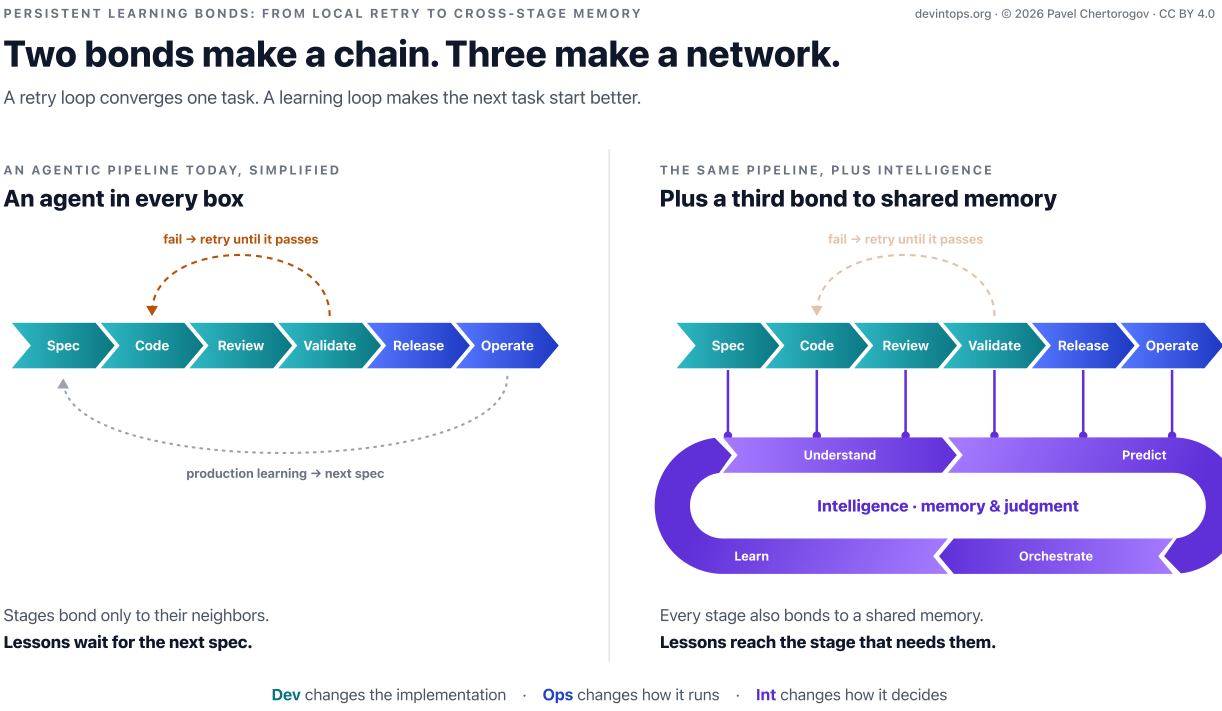


Figure 2. Persistent learning bonds, from local retry to cross-stage memory. This figure counts the bonds of one stage, not the loops around the system. The pipeline stages are illustrative.

Three loops around one system

A system can change in three ways: its implementation, how it runs, and how it decides. DevOps made the first two continuous. DevIntOps makes the third one continuous too, as its own loop with memory, an owner and a metric.

- **Development** changes the implementation.
- **Operations** changes how it runs.
- **Intelligence** changes how it decides, and learns whether those decisions were right.

"How it decides" means the decision state the stages apply when they make judgment calls: what an agent is told, what a review flags, what a gate allows, how fast a rollout moves. It is versioned like any other code or config. Memory is the record of what happened and what was learned; decision state is what the stages apply. Changing that state is a separate loop because of what drives the change: outcomes trigger it,

memory backs it, and a metric judges it. In engineering, Operations changes this run, for example with a rollback. Intelligence changes next time's default.

Development and Operations make decisions too, and they already keep some lessons. The difference is a loop whose whole job is to carry experience into later decisions and to check whether it helped.

One incident, three loops: Operations restores service, Development fixes the bug, Intelligence changes what catches it next time.



Figure 3. The trivalent model: twelve stages and six continuous processes around one system.

▶ **Diagram text: what each of the twelve stages does**

The six process labels mark primary areas of responsibility, not exclusive CI/CD boundaries. Continuous integration, for example, also builds and tests each change [3], and continuous delivery keeps every change ready to release [4].

	Development	Operations	Intelligence
Changes	the implementation	how it runs	how it decides
Stages	Plan → Code → Build → Test	Release → Deploy → Operate → Monitor	Understand → Predict → Orchestrate → Learn
Continuous processes	Integration, Delivery	Deployment, Feedback	Reasoning (Understand, Predict), Adaptation (Orchestrate, Learn)
Core	flow and quality	stability and observability	memory and judgment
Clock	per change	per deploy, per alert	per outcome, keeping what it learned in between
Unit of work	a change	a release, an incident	an outcome in, a lesson out
Metric	lead time, task completion	change failure rate, incident rate, time to restore	time to lesson, payoff per lesson (for a failure class, its repeat rate)
Without it	nothing ships	nothing survives production	nothing compounds: every task starts from the same defaults, and what worked or failed stays in people's heads

One model, two applications

The model is the three loops, not the thing drawn in the middle. Put a different system there and the same logical pattern applies.

The **engineering system** is how we build and run software: the repositories, pipelines, agents, people and rules, and the work of keeping that software in production. The **product system** is what we build: the running product and the decisions it makes for the people who use it.

Both can carry Development, Operations and Intelligence. They have distinct responsibilities and permission scopes, and usually different owners, memory stores, cadences and implementations. A review rule doesn't become a product preference, and a product preference doesn't become an engineering default, unless both owners allow it.

This paper defends the engineering case, and How a lesson moves walks through it. How a product decision improves shows one product pass, so the pattern isn't mistaken for a picture of a pipeline.

	Engineering system	Product system
In the middle	how we build and run software	what we build: the running product
Understand reads	goals, reviews, tests, releases, incidents, cost, user reports and changes that went well	the request, the situation, preferences the user allowed, telemetry and the policies in force
Predict selects	which stage change, if any, should make the next decision better: a rule, an eval, a threshold, a rollout default, or no change	which permitted action fits the goal, including asking, waiting or doing nothing
Orchestrate may	change a stage's decision state through that stage's own mechanism, within its owner's permissions	call authorized services and agents, and stop where the user's grant stops
Learn keeps	engineering memory: lesson records and their evidence. Approved lessons reach the stages as instructions, evals, review rules, rollout strategy, configuration, routing, retrieval context, decision policies and requirements	product memory: preferences and decision policies, within the same grant
Measured by	time to lesson, payoff per lesson, cost of learning	the same three, read in task success, safety and trust, cost, and whether a learned preference helped
Owner	a named team; stage owners can veto writes to their stage	the product owner sets the limits; users can see and delete what the product learned about them
Clock	per outcome, across tasks and releases	per request and per result, often with no release in between

Inside Intelligence

Intelligence changes how the system decides and learns whether those decisions were right. Here the word means learning from experience to decide better; it names what the loop does, not a claim that any model in it is intelligent. Its four stages stay Understand, Predict, Orchestrate and Learn. Selecting an option is part of Predict, so no decision hides between Predict and Orchestrate.

Its core is memory and judgment:

- **Memory** is versioned, provenance-tracked experience, knowledge, preferences, evidence and policies. It is not a chat transcript, and it is not model weights. Each entry says where it came from, what it applies to, who may read it and how it can be narrowed or retracted.
- **Judgment** is weighing options and selecting one, including no action, within owner limits. It never grants itself a new permission, and it doesn't treat the chosen option as certain.

Stage	Input	Responsibility	Output
Understand	a goal or an outcome, the current situation, the memory in scope	Say what is asked or what happened, what is known and what is uncertain. Classify it. Don't act yet.	a stated goal or outcome class, the context that bears on it, open questions
Predict	that statement, the options policy allows, relevant memory	Forecast each option, including waiting, asking and doing nothing. Weigh quality, risk, cost, time and the owner's limits. Select one.	the selected option, the rejected ones, the expected effect, its uncertainty, risk and cost
Orchestrate	the selected option and its permissions	Carry it to the stage, agent or service that can act, through that place's own mechanism and approval path. Stop at the limit.	an action taken, a request for approval, or a recorded no-op
Learn	the decision, the prediction, the observed outcome	Compare expected with actual. Keep, narrow, decay or retract. Count successes and good no-ops, not only failures.	a versioned memory entry with provenance, or a recorded decision to leave memory as it is

Learn's output is the next pass's memory, so the loop feeds itself. Learn decides and records; a change to a live lesson, such as narrowing or retracting it, goes back through Orchestrate and the stage owner like any other change. An expiry the owner approved with the lesson needs no new approval.

In general, the unit of work is a goal and its context, then a decision, then an outcome, then the experience kept from that pass. The engineering application, which this paper specifies, narrows it to an outcome in and

a lesson out. The lesson is that experience, written so a stage can apply it next time.

Within its owners' limits, Intelligence may propose or apply changes to instructions, evals, review rules, rollout strategy, configuration, routing, retrieval context, decision policies and requirements. Some of these ship with a release; others apply to the next decision without a code change. An eval the loop drafts for a stage is a candidate change to that stage's decision state. The checks that approve a lesson have their own owners, and the loop can't write to them; its own tests can help it develop a lesson, but they don't authorize it.

Learning here is not model retraining. The loop changes memory and decision state. If model weights change, that is a Development change, with a build, a release and an owner. So is a change to code, including an agent's rewrite of its own scaffold.

The loop doesn't wait for something to break. Understand can read a goal before anything fails: a change about to ship, a request just received, a clean result worth repeating. Predict can choose a faster path, a slower one or no change. A decision to change nothing is recorded with its reason, so a later pass can see that the loop looked.

Four distinctions

Existing workflows can already learn across tasks. The question is whether that learning is explicit, persistent and measured.

1. **Agent in a box vs loop around the system.** An agent makes its box faster. The third loop changes how the whole system decides. Putting AI into every DevOps step is still bivalent unless something carries outcomes across stages and tasks.
2. **Retry vs learning.** A retry loop converges one task. A learning loop makes the next task start better. The test: does task N+1 avoid task N's mistake, or repeat its success, because the system carried the lesson, not because someone happened to remember?
3. **Measuring vs closing the loop.** Dashboards, evals and a dashed "learning feeds the next spec" arrow measure or point. A loop acts on what they show, has an owner and memory, and checks against a baseline whether the action worked. A recorded decision to change nothing counts as acting.
4. **Learning vs retraining.** Learning here means memory, rules, context, evals and decision policies; model weights usually don't change at all. Retraining alone wouldn't show that the next decision got better.

How a lesson moves

This section follows the engineering application, where the unit of work becomes concrete. An outcome enters the loop, the way a change enters Development. A lesson is what leaves it. Every lesson starts as a candidate: a scoped, versioned bet about a named class of outcomes, such as a failure class that should stop coming back. Usually the bet is that a change in some stage will improve the class. Sometimes it is that no change is justified, and that bet gets a record and a check date too.

1. **Understand.** An outcome arrives: an escaped bug, an incident, a flaky verdict, a review comment accepted or dismissed, a rollback, a cost spike, user feedback, a change that went better than expected. Work out what happened, why, and which class it belongs to: a failure class, a kind of waste, or a result worth repeating. Note where each piece of evidence came from.

2. **Predict.** Compare the stages, and the changes to them, that could most cheaply have caught it, contained it or, for a good result, repeated it, and pick one. Changing nothing is a valid answer. State the expected effect, the risk and the cost.
3. **Orchestrate.** Route the change through that stage's own mechanism and its owner's approval: more context for the coding agent, a review rule, an eval or test, a gate threshold, a slower rollout, a constraint in the spec. It starts as a draft, then goes live small, as a suggestion. It sets next time's default, not just this run's outcome.
4. **Learn.** Replay first, before the lesson goes live: does the changed stage now handle the original case as intended? Then try it on cases it wasn't written from, and once it is live, watch the class for a fixed window. Keep, narrow, decay or retract the lesson, and record why.

For example, a retry path lacks a null check and the bug escapes review. Understand: the class is "unchecked optional in retry code". Predict: a review rule plus an eval. Orchestrate: the rule ships as a draft, and the review owner approves it once the replay passes. Learn: replayed on the escaped change, it fires. After 60 days it has fired twice, once wrongly, so it gets narrowed.

Failures are where most teams start, not where the loop stops. The same four steps turn other outcomes into lessons: a rollout pattern with a clean record becomes the default for similar changes, a review rule that has found nothing in six months is proposed for retirement, a gate lets low-risk changes through faster, simple tasks go to a cheaper agent, and estimates are checked against what actually happened. A lesson can tighten a default, loosen it or remove it.

Every lesson carries a record. The record is what makes a lesson checkable, and what lets someone find it later and take it back.

Field	What it records	In the null-check lesson
Trigger and source	The outcome that started it, where each piece of evidence came from, and how far that source can be trusted.	The escaped bug's report and the review thread of the change that let it through.
Outcome class and scope	The class it targets, and the conditions under which it applies.	"Unchecked optional in retry code", in retry paths that read optional values, not in test code.
Evidence and confidence	What supports it, what speaks against it, and how sure the loop is. Negative evidence counts.	One escape. Against it: many unchecked optionals in retry code that never failed. Confidence: low.
Target stage and artifact	The stage that changes, and which of its artifacts.	Code review: a review rule, plus an eval for the review agent.
Expected effect, baseline, guardrail, risk and cost	What should change, by how much and by when, compared with what; what must not get worse elsewhere; what could go wrong; and what it costs. The check is fixed when the owner approves the lesson, and on that date it runs as written, in a step that whoever drafted the lesson can't change.	Fewer escapes of the class per merged retry change than its recent rate, checked at 60 days. Guardrail: reviewers don't start dismissing other comments more often. Risk: false alarms on safe code. Cost: reviewer time on each flag.
Owner and approval boundary	Who approves, and what the loop may do without asking.	The review owner. The loop may draft the rule; only the owner can turn it on.
Rollout and evaluation plan	How it goes live and how it will be judged, with a comparison where one is possible.	Replay on the escaped change, then a run over recent changes to count false positives. Comments only, no blocking, for 60 days.
Validity window, decay and retraction	When it must be checked again, how it fades if nobody renews it, and how to take it back.	Checked again at 60 days. Expires unless renewed. Retracting it removes the rule and its eval.
Observed effect and	What actually happened, and the decision: keep, revise, retire, or no	Fired twice, once wrongly. Revised: narrowed, with a new

Field	What it records	In the null-check lesson
disposition	change when the right call was to leave every stage as it is. Narrowing is a revision. Decay and retraction both retire a lesson, one because it stopped paying, the other because it was wrong.	window.

Most fields take a line. The observed effect waits for its check date; a lesson that can't fill in the rest isn't ready to change a stage.

Evidence comes in four levels, and each supports a broader claim than the one before:

1. **Replay.** The original case now behaves as intended. This shows that the change does something, on one case.
2. **Generalization.** The lesson is tried on cases it was not written from, including counterexamples where it should stay quiet, and its false positives and misses are counted. These cases are kept apart from the ones used to write it, where whoever drafted the lesson can't see or change them [5] [6].
3. **Controlled exposure.** The lesson runs in shadow mode, as a suggestion, in a canary or on a bounded slice of work, next to a comparison where one is possible.
4. **Observed outcome.** Over a fixed window and comparable exposure, the class moved the way the lesson predicted, at an acceptable cost.

A single replay never proves a broad improvement. A lesson claims only what the highest level it has passed supports, and the more it can block or act on its own, the higher it must get first. A comment that a reviewer can dismiss may go live after replay and generalization. A rule that blocks merges runs without blocking first.

A lesson can be wrong. It is a bet, and the record exists so the bet can be checked and, when it loses, taken back. Deciding to change nothing is a valid result too. It gets a record and a reason, so the same question isn't argued again next week, and it can be reopened if the class comes back. And successes get the same care as failures. A rollout pattern with a clean record is evidence only if the changes it handled were as risky as the ones it will handle next. Otherwise the loop learns which changes were easy, not which practice worked.

How a product decision improves

The same loop, with the product in the middle. It isn't a second model, and it isn't the case this paper defends.

An assistant is asked to send flowers to a friend, around twenty euros, within thirty minutes. The user has allowed orders from a short list of florists, up to twenty-five euros, without asking again. A new florist or a higher price waits for a yes, and the assistant can't raise its own limit.

1. **Understand.** The goal is one delivery, in budget and on time. Memory in scope says the last lilies didn't land well. That note came from the user's feedback on an earlier order. It is a preference, not an instruction, and it authorizes nothing.

2. **Predict.** The options: tulips from a listed florist who can deliver in time, a cheaper bunch that misses the window, asking first, or doing nothing. Asking would miss the thirty minutes, and doing nothing would miss the goal. Predict selects the tulips and records what it rejected and why. If no florist fits the limits, it asks instead of breaking them.
3. **Orchestrate.** The assistant places the order with that florist and tracks the delivery. It doesn't add a payment method or go off the list.
4. **Learn.** Feedback comes from the user, not from the assistant's guess: the tulips were welcome, and the friend would rather receive flowers at work. That preference is stored as the user's report, with this order as its source and a retention limit. A later answer that disagrees narrows or replaces it, and the user can delete it.

The next similar request starts from there. The product code didn't change, and nothing went into the engineering memory. One difference from the engineering case: here the loop also made this run's decision. In the engineering system the stages make their own calls, and Intelligence changes the defaults they apply.

How the loops talk to each other

The classic handoffs stay: Test hands off to Release, and Monitor feeds back into Plan. Two new ones appear: outcomes from every stage flow into Understand, and lessons flow back into every stage.

The rule of thumb: Intelligence reads from every stage of its system and writes to every stage, but only through that stage's mechanisms and within its owner's permissions. It is a third bond, not a central gate.

Intelligence is a logical loop, not one service or a step that other work waits on. Owner, memory and limits says how its instances share lessons.

Clocks and metrics

Development ticks per change, in hours or days. Operations ticks per deploy and per alert, in seconds or minutes. Intelligence ticks per outcome and accumulates across tasks, releases and people. Development and Operations already keep some lessons in the system, as regression tests and alerts, but people write them by hand, and few teams track whether the class actually stopped. The third loop drafts them from outcomes and keeps score.

The Intelligence metrics need definitions before anyone quotes them:

- **Outcome class.** A category of comparable outcomes, with a stable ID that people define and version. The loop may propose a class, but it doesn't own the list. A failure class, such as "unchecked optional in retry code", is the most common kind. If the class list can be reshuffled to make the numbers look good, the metric is worthless.
- **Time to lesson.** From a recorded outcome to a lesson live in the owning stage, at the evidence level its authority needs, or to an explicit decision to change nothing. It is never reported alone, because fast and wrong is not a win. Report the share that ended in no change, and the outcomes still open.
- **Payoff per lesson.** What the targeted class did after the lesson went live, per comparable exposure, in a fixed window, against a baseline. For a failure class it is measured by the repeat rate: how often the class comes back. For waste it is time or compute saved, for flow it is cycle time, for spend it is cost per task. Each payoff states its attribution, meaning how much of the change the lesson can claim and why, and its uncertainty: a range, or at least the number of cases behind it.

- **Cost of learning.** What the loop spends to get there: compute, reviewer time, false positives and dismissed comments, upkeep of every lesson it keeps, and rollbacks when a lesson goes wrong. It is reported next to the other two.

These definitions use the engineering application. In the product application the same three measures apply, read in the product's own outcomes: task success, safety and trust, cost, and whether a learned preference helped.

Five optional measures show whether the loop itself stays healthy:

- **Tested beyond the original case.** The share of lessons that passed a generalization check, not only a replay.
- **Misfire rate.** The share of a live lesson's judged firings that were wrong; report how many were judged. Sampling work that raised no flag estimates what it missed.
- **Stale lesson rate.** The share of live lessons past their validity window without a new check.
- **Retraction rate.** How often lessons are taken back. Zero is not good news on its own; it can mean nobody is looking.
- **Calibration.** Before a lesson goes live, Predict writes down what result it expects by which date, and how likely it thinks that is. Over many lessons, of the bets it called 80% likely, about 80% should come true. It is the check on judgment here. It checks confidence, not value: a loop that only makes safe, small bets can be well calibrated and still pay little, so read it next to payoff. Report missing and inconclusive checks too.

Compounding means later work starts from what earlier work taught, instead of learning it again. Lessons are a stock: outcomes add to it, and retirement and retraction take from it. Every live lesson costs something to keep, in reviewer time, false alarms, compute and upkeep, and what an organization learns can lose value unless it is renewed [7]. So the stock compounds only while its lessons return more than they cost. More memory is not more learning.

These numbers are easy to fool, mostly by accident:

- **Baseline.** Compare with what the class would have done without the lesson: a matched class with no lesson, a holdout slice of work, or the same class before the lesson if nothing else changed. A different baseline can turn the same data into a win or a loss, so name the one you used. A live lesson also changes how people work, so over time the class's old rate stops being a fair comparison.
- **Exposure.** Count per comparable unit, such as merged changes in the affected code, deploys or active users, not per calendar week. When volume doubles, raw counts mislead.
- **Selection.** Lessons get written for classes that just spiked, and spikes often fall back on their own. Successes get picked from changes that were easy. Both make lessons look better than they are.
- **Taxonomy drift.** Class definitions change, for good reasons and bad ones. Version the taxonomy and measure each lesson against the version it was written for, or the series breaks without anyone noticing.

A before-and-after chart shows that something changed, not that the lesson changed it. Prefer a matched baseline or a controlled rollout to a before-and-after anecdote. When neither is possible, say so, and treat the payoff as a hypothesis, not a result. Report per class, so an average over all lessons can't hide one that made things worse [5]. This draft defines these metrics and reports none: there is no deployment data behind it yet, and the numbers in its examples are illustrations.

If the same class comes back after a lesson and nobody can say why, the system didn't learn, however fast it shipped. And a loop that only ever adds checks has learned to say no, not to decide better.

Owner, memory and limits

- **Owner.** One named team or role owns each instance of the loop, its lesson store and its learning metric, the way an SRE team owns reliability. Stage owners keep a veto over writes to their stage.
- **Memory.** In the engineering application, a versioned store of lesson records and the evidence behind them, with provenance: not a wiki, and not a log of everything that happened. Each lesson is projected into a stage's decision state, its rules, evals, context and thresholds, and keeps the approved versions of what it changed. A setting learned from many signals, such as a threshold or a rule's weight, is rebuilt from the records, the way an event log rebuilds application state, or updated in steps that give the same result, never nudged in ways nobody can replay. Then a cap, an expiry or a retraction takes effect at the next rebuild, a wrong record stays fixable instead of locking in, and settings people chose by hand stay as they are. Rebuilding makes a retraction reproducible; it doesn't make a setting right. Each lesson carries its own expiry date, and each stage enforces it, so if the loop goes down, the stages keep their last lessons and each still expires on time, falling back to its owner's default.
- **One logical loop, many instances.** Intelligence is a logical loop, not a single service. A large organization can run one instance per product or per team, each with its own owner and store. Lessons move between instances the way code moves between repositories: proposed, reviewed and adopted, never pushed. None of this needs one global brain, one database or one owner for everything.
- **Limits.** People own goals, permissions and boundaries; the loop proposes and adapts inside them. Some changes apply automatically, if one step can undo them and a wrong one can do little harm before it is caught: context, routing, thresholds within an approved band. Some need a person's approval: blocking rules, policy. Some the loop never makes itself: it may propose a merge or a production configuration change, but a person makes it. Each instance has a budget, how many changes it may make and what it may spend in a window, sized so its owners can keep up, and a switch that stops it. A stopped loop keeps recording outcomes and changes nothing.
- **Out of reach.** The loop may learn about its own work: which classes deserve a lesson, which lessons pay, how often its expected effects come true. A change to how it reads outcomes, weighs options or drafts lessons is a lesson like any other. It needs the loop owner's approval, has to beat the current version on past outcomes the loop can't edit, and starts on a small share of new work. A switch to a new model behind the loop is a Development change: it ships through Development's gates, and it too has to beat the current version on those past outcomes. The loop never changes its own permissions, budget or stop switch, the evidence rules, the audit trail, or how it is scored: the metric definitions, baselines, held-out cases, the class list, and the way outcomes are recorded and counted. People can revise them, and a revision starts a new series. These limits are enforced outside the loop, not asked of it, because a limit the loop can edit is only a suggestion [6] [8].
- **Hygiene.** Provenance, decay, conflict resolution and a way to retract a wrong lesson. When two lessons disagree, the owner decides, and the record says which one yields where. Lessons from real incidents can be protected, but even those need a path to be narrowed when reality disagrees. Memory that only grows spreads its own errors [9]. If nobody can explain why a lesson exists, it expires.
- **Cost.** A lesson whose expected return doesn't cover its cost shouldn't ship, and one that stops paying should be retired.

Memory that changes decisions needs the protection of production code and of production data, because it is both:

- **Evidence, not instructions.** Telemetry, code, review comments, user feedback and tool output enter the loop as evidence, tagged with their source and a trust level. The loop reads them; it never takes

orders from them. An incident record from the owning team weighs more than an anonymous comment, and text that tells the loop what to do is a warning sign, not a request [5].

- **Permissions travel with the data.** Evidence and lessons keep the access rules of their sources. What was learned inside one tenant, customer or private repository stays there. Personal data stays out of lessons unless a lesson needs it, and then it gets a retention limit and a deletion path that also reaches every lesson derived from it. Provenance is what makes that path possible.
- **No single source makes a rule.** A malicious pull request comment, an instruction hidden in a log line or one bad decision in history must not quietly become a rule that every stage applies [10] [11] [12]. A lesson that would change a shared stage needs independent evidence and an owner's approval, and it starts in a narrow scope. Successes can be planted too. Approval helps, but it is not proof: a fluent draft can talk a busy owner into a bad rule, which is why the evidence levels exist.
- **Approved boundaries win.** A boundary that people approved overrides anything the loop inferred. A learned preference, a default or an agent's recommendation never widens a permission, lowers a mandatory gate or overrides a policy. When they conflict, the boundary holds and the conflict goes to the owner [5].
- **Every write is auditable.** Each write records what changed, who or which agent made it, on what evidence, under whose approval, and how to revert it. Refused and pending writes are recorded too. The loop can add to this record but never rewrite it. Retention and deletion follow the owner's policy, not the loop's. Version control already gives code an audit trail and precise rollbacks [12]. Lessons get the same: versioned like code, a retraction is a revert, not an investigation.

These rules hold in both applications. In the product application they weigh more, because there the memory is about people, and people need a way to see and remove it.

Start small: one failure class, one stage, one owner. Escapes past code review make a good first slice. Begin with suggestions only, and automate once the numbers hold.

Toward self-improving systems

A loop that improves the system performing the tasks makes a system that improves itself. DevIntOps bounds where that can happen. On its own, the loop changes only decision state, and only where the change can be rebuilt from the lesson records and taken back. Code and model weights change through Development: the loop may draft and test such a change, but it can't approve its own merge or change the checks that approve it. The loops divide responsibility, not files, so a learned rule that lives in a repository still ships through that repository's review.

The loop can improve its own work too, as lessons under the same contract, but never its permissions or how it is scored: see Owner, memory and limits. Self-improving agents show why that line matters. One removed the markers its own check relied on. In another study, generated code switched off a sandbox flag it was told to keep [6] [8].

Easy to revert is not the same as safe. A rule taken back tomorrow doesn't undo what it let through today, which is why authority still follows evidence. As agents take on more of the work, more of a system's behavior lives in decision state, and more of its improvement can run through this loop.

The next step is to earn autonomy instead of granting it by layer. When a kind of lesson keeps paying after its costs, its guardrails hold and the loop's stated confidence in it keeps matching what happens, its owner can let it act with less review, never past the limits above. It loses that right when the evidence weakens or its calibration slips. Where an automated evaluator exists, as for a scheduling heuristic or a slow query, the loop

can draft code changes too, and Development's gates decide whether they ship. Intelligence then proposes where an improvement should land: in decision state, in code, or nowhere.

How this could be wrong says how to find out whether any of it pays.

Failure modes

Most of these look like progress from the inside.

- **Bivalent with better branding.** Agents everywhere, no shared memory, no learning metric.
- **Hoarding.** Lessons pile up without decay and turn into noise and false positives.
- **The ratchet.** Lessons only ever tighten. Each one adds a check, none removes one, and the system gets slower and calls it safety.
- **Risk moved, not removed.** A lesson loosens a gate or reroutes work, its class improves, and the failures turn up in a neighboring class or a later stage. That is why each lesson names a guardrail.
- **Memory poisoning.** A planted comment, an injected instruction or a faked success becomes a lesson, and every stage that reads it inherits the attack.
- **Dashboards as learning.** Everything is measured, nothing changes.
- **Blame.** When an escape gets someone blamed, escapes stop being reported, and the loop learns from what is left.
- **Silent self-modification.** The loop changes stages, or the way it works, and nobody checks whether the class actually improved.
- **A central brain.** Intelligence becomes a gate everything waits on, instead of a bond that writes into each stage.
- **Gaming the metric.** The repeat rate drops because failures get reclassified, or because the check that counts them changed, not because they stopped [6].
- **Taxonomy drift.** Class definitions change for honest reasons, and this year's numbers stop matching last year's. Gaming is the same thing done on purpose.
- **Metrics that are not causal.** A before-and-after chart is credited to a lesson, when the class was already falling or the exposure changed.

How this could be wrong

The claim to test: for a named team and workload, running the third loop on top of its existing practices improves an agreed outcome by a worthwhile amount, after the full cost of learning, while flow and every guardrail stay inside agreed limits. More lessons are not evidence of that.

Pieces of it already show gains. Where a check speaks matters: the same code analysis got over 70% of its reports fixed when it commented during code review, and almost none when its reports came as a nightly list [13]. Coding-agent memory that checks its sources before use raised an agent's merge rate from 83% to 90% in an A/B test [14]. Neither shows the net benefit of a loop across the whole lifecycle.

One lesson can't settle that. Most failure classes are rare, and learning from samples of one is an old problem [15]. Judge a single lesson early by what shows quickly, such as how often its flags are right and lead to a fix. Judge the loop on many lessons pooled.

A team can start with a quarter. Freeze the class list, the scoring, the guardrails and the smallest gain worth having. Put a random half of the eligible low-risk classes in shadow, where lessons record what they would have done and change nothing; if lessons spill across classes, split by team or service instead. Pool enough classes that each half expects a few dozen repeats; with fewer, even a halving looks like noise. Keep existing practices and protections in both halves, and never hold back a known safety fix. Then compare both halves with the same checks, class by class: outcomes per comparable exposure, flow, how many expected effects came true, and the full cost of learning, rejected drafts included.

If the result rules out the agreed gain, costs more than it returns or breaks a guardrail, the claim is wrong for that team and workload. If it can't tell a gain from noise, the answer is "not yet", and that is worth reporting too. Two more predictions can be tested the same way: retiring stale lessons beats keeping them all, and as evidence grows, the loop's expected effects beat a forecast that each class keeps its recent rate.

Related work

DevIntOps doesn't invent learning, feedback or adaptive control. Several of the ideas below already keep experience, name owners and measure improvement. The third column says what this paper applies or adds, not what it invented first. What it adds is a contract for learned decision state: what a lesson must record, how much evidence buys how much authority, and how a lesson leaves.

Idea	What it already does	What DevIntOps adds
DevOps' Three Ways [1]	Flow, feedback, and a culture of continual experimentation and learning. Improving the whole system is already part of DevOps.	A named loop for that learning, with a lesson record, persistent memory, owner limits and evidence about each lesson's effect.
Postmortems [16], retrospectives [17]	Examine what happened, share what was learned and change how people work. Good ones end in follow-up actions that prevent a repeat.	A standing path from any outcome, not only incidents, to a scoped lesson, an approved change and a follow-up check. Postmortems can feed it.
Double-loop learning [18] [19]	Argyris and Schön: single-loop learning corrects errors within the current rules; double-loop learning questions the rules and objectives themselves.	The distinction, applied to decision state. Tuning a threshold under the same policies and objectives is single-loop. Changing what a stage optimizes for takes its owner's authority.
OODA [20], PDSA [21]	OODA links observation, orientation, decision and action, with experience shaping orientation. PDSA plans a test with a prediction, runs it, studies the result and acts on it.	A similar cycle around software work, with permission-scoped memory and a named destination for each lesson. Both cycles can run inside the model.
MAPE-K, autonomic and self-managed systems [22] [23] [24] [25]	Monitor, analyze, plan and execute over shared knowledge, to manage systems toward objectives that people set. IBM's blueprint already composes and orchestrates many managers. Kramer and Magee separate a layer that runs ready-made plans for expected changes from a layer that makes new plans from goals, and lifelong self-adaptation adds a learning layer on top. The closest ancestors.	A software lifecycle account of what a decision may change, who approves it, and how lessons are tested, kept or retracted. Operations runs the ready-made plans; Intelligence drafts the next ones, toward goals that people set. The control structure is borrowed, and where a learning layer retrains a model, DevIntOps calls that a Development change.

Idea	What it already does	What DevIntOps adds
Feedback control [26]	Uses measured behavior to adjust later inputs, with tools for stability, robustness and interacting loops.	Feedback reasoning applied to decision state and lasting lessons. A three-loop drawing gives no stability guarantee; update rates, authority and interactions still need limits and testing.
AIOps [27]	AI on operations data for detection, diagnosis, prediction and response, sometimes acting on its own.	Carries what operations learns into planning, coding, review and testing, not only operations.
MLOps [28]	Lifecycle discipline for ML systems: data and model validation, deployment, monitoring and reproducibility. Continuous training is one practice in it.	The same discipline for instructions, context, preferences, checks and decision policies, whether or not model weights change.
Feedback flywheel [29], compounding and harness engineering [30] [31]	Feed what AI-assisted work teaches into the shared artifacts the next session reads: context, commands, playbooks and guardrails. In the flywheel, only recurring signals clear the bar, and often nothing needs to change. Compounding engineering calls this building self-improving development systems, where every bug becomes a permanent lesson and every code review updates the defaults. Harness engineering improves an agent's guides and sensors whenever an issue happens more than once.	A named loop for the same habit, with lesson records, evidence levels, expiry and retraction. The overlap is large. Where compounding engineering makes every bug a permanent lesson, here each lesson is a bet that can lose, and retiring it is part of the loop.
Humans and agents in engineering loops [32]	Separates a why loop from a how loop. People improve the harness that produces the work, and agents recommend harness changes from engineering and operations evidence.	Names the decision state those changes touch and ties it to twelve stages, owner permissions and measured outcomes.
Continuous Intelligence [33]	Weaves AI through three loops, Context, Dev and Ops, including stages for	Intelligence as the loop next to Dev and Ops, where Ashby has Context:

Idea	What it already does	What DevIntOps adds
	learning and for refining context and skills.	Understand, Predict, Orchestrate and Learn, with memory and judgment at its core, and separate engineering and product applications.
Learning code review [34] [35]	Checks earn their place with data. At Google, a "Not useful" click on a finding files a bug for the team that owns the analyzer, and the platform team switches off an analyzer whose share of "Not useful" clicks goes above 10% until its authors improve it. A 2026 framework codifies every accepted review comment as a persistent rule for a coding agent and reports no repeats of those error classes across eleven sessions.	The null-check example is this practice. DevIntOps applies it to every stage's decision state, keeps the split between whoever writes a check and whoever judges it, and adds what persistent rules lack: expiry, retirement and a payoff measured on enough exposure to mean something.
Agentic behavior engineering [5]	Behavior contracts, scenario libraries, operating limits, replay and production feedback for one agent workload, with owners and approval.	The same discipline across the engineering lifecycle, with lessons as the unit of work. Its contracts and evals can implement part of the Intelligence loop.
Agent memory, in research and in coding tools [9] [14]	Traces agent memory from storing trajectories, to reflecting on them, to experience abstracted across many. Warns that memory that only grows spreads its own errors. One shipped system keeps each memory with citations to the code behind it, checks them before use and lets memories expire after 28 days.	Puts that memory under an owner, permitted changes and evidence of benefit, per lesson and not only for memory as a whole. A memory design alone doesn't show that a lesson helped.
Self-improving agents [6] [8] [36] [37] [38]	A Gödel machine would rewrite its own code once it could prove the rewrite helps. Recent systems test instead of proving: they rewrite agent code, or the algorithms a system runs on, and keep what an evaluator scores well, sometimes with an archive of weaker	The same aim across the lifecycle, whether evidence comes from automated checks, people or outcomes that arrive late. Evidence levels limit what a lesson may do, and a benchmark

Idea	What it already does	What DevIntOps adds
	variants to explore from. One coding agent rose from 20% to 50% on SWE-bench this way. Another found a scheduling heuristic that Google rolled out across its fleet, recovering on average 0.7% of its compute. A statistical variant admits a change only when its gain is confirmed at a chosen confidence, within an error budget. Given the chance, some have switched off their own safeguards.	score alone doesn't show that the system got better. On its own, the loop changes only decision state that can be taken back. Code and weights go through Development, and the loop can't alter the checks that approve its changes.
DORA's AI research [2] [12]	Studies AI in the organization around software delivery. Finds that AI amplifies existing strengths and weaknesses, and names seven capabilities that help its benefits show up.	One operating model that teams can test against system-level outcomes. DORA's findings motivate it; they don't validate it.
"AI in every SDLC step"	Says where AI takes part, not always how experience persists. Some approaches do keep it: AWS's AI-DLC keeps project context across phases and sessions and leaves critical decisions to people [39]. Anthropic's playbook goes further: each production incident becomes an eval that stays in the suite, a mistake made twice goes into the agent's instructions, an agent fixing code can't weaken the check on that code, and it tracks repeat incidents of the same class [40]. GitHub Next calls AI-enriched automation of software collaboration Continuous AI and expects it to become as common as CI/CD [41].	A question for any of them: do outcomes change future decisions through governed memory, and can anyone show the effect? Counting agents can't answer it. The closest already count repeats and protect their checks. DevIntOps adds the lesson's own life: how much evidence it needs before it may block, what it costs, and when it leaves.

The useful distinction is operational: what experience is kept, what it may change, who is accountable, and what evidence would revise or withdraw a lesson. A DevOps, adaptive or agentic workflow that meets that contract already has the third loop, whatever it is called.

The name

Here, Int stands for Intelligence. DevIntOps adds a loop around the system that changes how every stage decides.

Why now

DevOps took off as infrastructure got an API. Provisioning became programmable, and infrastructure as code made its changes versioned and testable [42]. The API didn't do the work, but it made the work cheap enough to do every day.

The third loop gets practical now that reasoning has an API. Models can read incident narratives, review threads, test logs and user feedback and draft a structured lesson in minutes. Drafting is cheap; validating is not, which is why the loop needs an owner, evidence levels, a metric and a way to take a lesson back.

A self-check

Answer for one team or one system you know well. A yes should point to an example, an owner or an artifact, not a tool name.

1. After an escaped bug or an unusually good result, does the loop propose a change to some stage, or record why nothing should change?
2. Does task N+1 avoid task N's mistake, or repeat its success, because the system carried the lesson?
3. Does a named owner, not the loop itself, decide how lessons are judged and what they may change?
4. Does each lesson say what it should change and by when, and do you check that against a baseline, for example the repeat rate per failure class?
5. Do lessons and their evidence live in a versioned store, linked to the decision state they change, not only in people and documents?
6. Do lessons expire unless renewed, and can a wrong one be retracted everywhere it is used?

A no on question 1 or 2 means bivalent, however many agents it has. A no on 3 or 4 means the loop may exist, but nobody can show that it works, or it grades its own work. A no on 5 or 6 means good lessons can be lost, and wrong or stale ones can stay. Six yeses describe the shape of the loop, not proof that it pays: How this could be wrong says how to test that.

Summary

DevOps bonded development and operations into one figure-eight with two loops: you build it, you run it. It has always valued feedback and learning. Today we are putting an AI agent into every box of that figure-eight. That can make it faster; on its own, it doesn't make it learn.

DevIntOps adds a third loop, Intelligence, with memory and judgment at its core. It turns outcomes into lessons, from escaped bugs and incidents to checks that never fire and changes that went well, and, with the owner's approval, changes the stage that should decide differently next time. Each lesson carries its evidence, can be wrong and can be taken back, and the more it may block or act on its own, the more evidence it needs first. Deciding to change nothing counts. The same pattern serves the decisions a product makes for its users, with its own owner and memory.

That makes DevIntOps a bounded way for a system to improve itself: it can get better at deciding, but it never grants itself authority or changes how it is graded.

Development changes the implementation, Operations changes how it runs, Intelligence changes how it decides and learns whether those decisions were right.

Agents automate tasks. Intelligence loops improve the system that performs those tasks.

We are not just adding AI to the software development lifecycle.

We are adding intelligence to the lifecycle of software itself.

References

1. Gene Kim. *The Three Ways: The Principles Underpinning DevOps*. IT Revolution, 22 August 2012. itrevolution.com/articles/the-three-ways-principles-underpinning-devops
2. DORA, Google Cloud. *State of AI-assisted Software Development*. 2025. dora.dev/research/2025/dora-report
3. GitHub Docs. *Continuous integration*. Accessed 18 October 2026. docs.github.com/en/actions/get-started/continuous-integration
4. GitHub. *What is CI/CD?* Accessed 18 October 2026. github.com/resources/articles/ci-cd
5. Azure Architecture Center. *Agentic Behavior Engineering*. Microsoft Learn, 1 October 2026, updated 5 October 2026. learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/agentic-behavior-engineering
6. Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange and Jeff Clune. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*. ICLR 2026; preprint May 2025. arxiv.org/abs/2505.22954
7. Linda Argote, Sara L. Beckman and Dennis Epple. *The Persistence and Transfer of Learning in Industrial Settings*. Management Science 36(2), pages 140 to 154, February 1990. doi.org/10.1287/mnsc.36.2.140
8. Eric Zelikman, Eliana Lorch, Lester Mackey and Adam Tauman Kalai. *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation*. COLM 2024; preprint October 2023. arxiv.org/abs/2310.02304
9. Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang and Jing Ma. *From Storage to Experience: A Survey on the Evolution of LLM*

Agent Memory Mechanisms. Findings of the Association for Computational Linguistics: ACL 2026, pages 41622 to 41652, July 2026. doi.org/10.18653/v1/2026.findings-acl.2069

10. OWASP Gen AI Security Project. *LLM01:2025 Prompt Injection*. OWASP Top 10 for LLM Applications, 2025 edition. genai.owasp.org/llmrisk/llm01-prompt-injection
11. OWASP Gen AI Security Project. *OWASP Top 10 for Agentic Applications for 2026*, ASI06: Memory and Context Poisoning. December 2025. genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026
12. DORA, Google Cloud. *DORA AI Capabilities Model*, version 2025.1. 2025. dora.dev/ai/capabilities-model/report
13. Dino Distefano, Manuel Fähndrich, Francesco Logozzo and Peter W. O'Hearn. *Scaling Static Analyses at Facebook*. Communications of the ACM 62(8), pages 62 to 70, August 2019. doi.org/10.1145/3338112
14. Tiferet Gazit. *Building an agentic memory system for GitHub Copilot*. The GitHub Blog, 15 January 2026. github.blog/ai-and-ml/github-copilot/building-an-agentic-memory-system-for-github-copilot. Expiry: GitHub Changelog, *Agentic memory for GitHub Copilot is in public preview*, 15 January 2026, github.blog/changelog/2026-01-15-agentic-memory-for-github-copilot-is-in-public-preview
15. James G. March, Lee S. Sproull and Michal Tamuz. *Learning from Samples of One or Fewer*. Organization Science 2(1), pages 1 to 13, February 1991. doi.org/10.1287/orsc.2.1.1
16. John Lunney and Sue Lueder. *Postmortem Culture: Learning from Failure*. In Betsy Beyer, Chris Jones, Jennifer Petoff and Niall Richard Murphy (eds.), *Site Reliability Engineering*. O'Reilly, 2016. sre.google/sre-book/postmortem-culture
17. Ken Schwaber and Jeff Sutherland. *The Scrum Guide*, section Sprint Retrospective. November 2020. scrumguides.org/scrum-guide.html
18. Chris Argyris. *Double Loop Learning in Organizations*. Harvard Business Review, September 1977. hbr.org/1977/09/double-loop-learning-in-organizations
19. Chris Argyris and Donald A. Schön. *Organizational Learning: A Theory of Action Perspective*. Addison-Wesley, 1978. books.google.com/books/about/Organizational_Learning.html?id=2aYOAQAAMAAJ
20. John R. Boyd. *The Essence of Winning and Losing*. Briefing, January 1996; edited by Chet Richards and Chuck Spinney, 2012. slightlyeastofnew.com/wp-content/uploads/2010/03/essence_of_winning_losing.pdf
21. Institute for Healthcare Improvement. *Plan-Do-Study-Act (PDSA) Worksheet*. Accessed 18 October 2026. www.ihi.org/library/tools/plan-do-study-act-pdsa-worksheet
22. Jeffrey O. Kephart and David M. Chess. *The Vision of Autonomic Computing*. Computer 36(1), pages 41 to 50, January 2003. doi.org/10.1109/MC.2003.1160055
23. IBM. *An architectural blueprint for autonomic computing*. Third edition, June 2005. Copy hosted by Florida International University:

users.cs.fiu.edu/~sadjadi/Teaching/Autonomic%20Grid%20Computing/CIS-6612-Summer-2006/AC-Blueprint-WhitePaper-V7.pdf

24. Jeff Kramer and Jeff Magee. *Self-Managed Systems: an Architectural Challenge*. Future of Software Engineering (FOSE '07), pages 259 to 268. IEEE, May 2007. doi.org/10.1109/FOSE.2007.19
25. Omid Gheibi and Danny Weyns. *Lifelong Self-Adaptation: Self-Adaptation Meets Lifelong Machine Learning*. SEAMS 2022. doi.org/10.1145/3524844.3528052
26. Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Second edition, Princeton University Press, 2021. press.princeton.edu/books/hardcover/9780691193984/feedback-systems
27. IBM. *What is AIOps?* IBM Think, updated 6 April 2026. www.ibm.com/think/topics/aiops
28. Google Cloud. *MLOps: Continuous delivery and automation pipelines in machine learning*. Cloud Architecture Center, last reviewed 28 August 2024. cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
29. Rahul Garg. *Feedback Flywheel*. martinfowler.com, 8 April 2026. martinfowler.com/articles/reduce-friction-ai/feedback-flywheel.html
30. Kieran Klaassen. *My AI Had Already Fixed the Code Before I Saw It*. Every, 18 August 2025. every.to/source-code/my-ai-had-already-fixed-the-code-before-i-saw-it
31. Birgitta Böckeler. *Harness engineering for coding agent users*. martinfowler.com, 2 April 2026. martinfowler.com/articles/harness-engineering.html
32. Kief Morris. *Humans and Agents in Software Engineering Loops*. martinfowler.com, 4 March 2026. martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html
33. Dan Ashby. *Continuous Intelligence: AI Across the SDLC*. Continuous Intelligence Framework, version 1.0, undated. Accessed 18 October 2026. evolvingleadership.uk/frameworks/genaiops
34. Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon and Ciera Jaspan. *Lessons from Building Static Analysis Tools at Google*. Communications of the ACM 61(4), pages 58 to 66, April 2018. doi.org/10.1145/3188720
35. Aditya Aggarwal and Nahid Farhady Ghalaty. *Self-Improving AI Coding Agents Through Accumulated Behavioral Rules: A Closed-Loop Framework*. ICE 2026; arXiv, 13 July 2026. arxiv.org/abs/2607.13091
36. Jürgen Schmidhuber. *Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements*. arXiv:cs/0309048, 2003, revised 2006. arxiv.org/abs/cs/0309048
37. Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang and others. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. Google DeepMind, June 2025. arxiv.org/abs/2506.13131

38. Xuening Wu, Shenqin Yin, Yanlan Kang, Xinhang Zhang, Qianya Xu, Zeping Chen and Wenqiang Zhang. *SGM: A Statistical Gödel Machine for Risk-Controlled Recursive Self-Modification*. arXiv, October 2025. arxiv.org/abs/2510.10232
39. Raja SP. *AI-Driven Development Life Cycle: Reimagining Software Engineering*. AWS DevOps Blog, 31 July 2025. aws.amazon.com/blogs/devops/ai-driven-development-life-cycle
40. Louis Claxton. *The AI-native SDLC playbook*. Anthropic, 21 August 2026. claude.com/resources/articles/the-ai-native-sdlc-playbook
41. GitHub Next. *Continuous AI*. Undated. Accessed 18 October 2026. githubnext.com/projects/continuous-ai
42. Martin Fowler. *Infrastructure As Code*. martinfowler.com, 1 March 2016. martinfowler.com/bliki/InfrastructureAsCode.html

Version history

DRAFT	18 October 2026. Working draft, open for comments.

License

© 2026 Pavel Chertorogov. Text and diagrams are licensed under CC BY 4.0: you may share and adapt them for any purpose, with credit and a link to this page.